Interview Guide: AWS Student Builder Group Co-Head

How to present Tasks 1 and 3, and how to answer follow-up questions on cloud fundamentals, DevOps and troubleshooting.

The panel scores **understanding and reasoning, not memorised definitions**. Every section below explains the *why*, so you can rebuild any answer from first principles if a question is phrased differently.

Criterion	Weight	Where this guide prepares you
Cloud infrastructure and service knowledge	30%	§3, §5
Architecture and technical reasoning	25%	§3, §8
DevOps and deployment knowledge	20%	§4, §6
Troubleshooting and problem-solving	15%	§7
Communication	10%	§1, §2

---

1. Why these two tasks

> "I chose Task 1 and Task 3 because they connect. Task 1 designs a system, and Task 3 is how that system gets built, shipped and kept healthy. Instead of writing about them separately, I built one repository where the architecture from Task 1 is real code, deployed by the pipeline from Task 3. So I can show you not just *what* I'd do but that it actually works."

Why not Task 2: it's mostly service descriptions. Tasks 1 and 3 cover the same services, but in the context of *decisions*, which is what the rubric rewards.

Be upfront about scope. The infrastructure is written in Terraform and validated, but **not deployed**, because the guidelines say not to buy cloud services for recruitment. Everything that can run without AWS was run and tested:

- 17 unit tests
- the production Lambda image tested with AWS's emulator
- a concurrency test against DynamoDB Local
- the website clicked through in a browser

Saying this clearly is a strength, not a weakness.

---

2. Two-minute pitches (practise out loud)

Task 1 pitch (≈ 2 min)

> "The problem is a registration site whose traffic is spiky: near zero most of the year, then a few thousand students in five minutes when the link is shared. So I chose a **serverless** design on AWS in the Mumbai region.
>
> Students hit **CloudFront**, which serves the static site from a private **S3** bucket and forwards `/api` calls to **API Gateway** and **Lambda**. Data lives in **DynamoDB** on-demand.
>
> The interesting problem is **overbooking**: if 50 people click for the last 3 seats at once, a naive read-then-write gives away more than 3. I solved it with a single DynamoDB **transaction**: increment the seat count only if seats remain, and insert the registration only if that email hasn't registered. I tested it: 50 concurrent requests, exactly 3 succeeded.
>
> Emails go through an **SQS** queue to a worker, so a slow email service never slows down registration, and failures retry and land in a dead-letter queue.
>
> Security is layered: **WAF** and HTTPS at the edge, a private bucket, **Cognito** login for organisers, least-privilege **IAM** roles and no stored keys.
>
> For availability, every service is multi-AZ by default, and a bad release can be rolled back in seconds by moving a Lambda alias.
>
> The main trade-off is that serverless costs almost nothing when idle and scales instantly, but has cold starts, a 15-minute limit and more AWS lock-in. For this workload that trade is clearly worth it."

Task 3 pitch (≈ 2 min)

> "Here's how a change gets from my laptop to students.
>
> I work on a **Git branch** and run the *same container image* locally with `docker compose`, with DynamoDB Local standing in for AWS. I open a **pull request**.
>
> **GitHub Actions** runs CI on every push:
>
> - linting
> - 17 unit tests with fake AWS and an 80% coverage gate
> - building the Docker image
> - smoke-testing that exact image with AWS's Lambda emulator
> - a vulnerability scan with Trivy
> - a 50-user rush test
> - Terraform format and validate
>
> Nothing merges unless it's green.
>
> On `main`, after an approval, the **CD** job gets temporary AWS credentials through **OIDC**, so there are no stored keys. It pushes the image tagged with the commit SHA, publishes a new Lambda version and smoke-tests it *before* it gets traffic. Only then does it switch the `live` alias. If anything fails, it automatically switches back.
>
> All infrastructure is **Terraform**, so it's reviewable and reproducible.

> In production, **CloudWatch** has JSON logs with request IDs, metrics, a dashboard and alarms on what students feel: errors, latency, throttling and lost emails. Alarms email the on-call person through **SNS**.
>
> And a real example of why testing matters: my tests caught a bug where `capacity` is a reserved word in DynamoDB. Without the tests, every registration would have failed in production."

---

3. Task 1 deep dive: every component, why it's there, what else I considered

Request journey:

1. Route 53
2. CloudFront (+ WAF)
3. S3 (static files) *or* API Gateway → Lambda
4. DynamoDB
5. SQS → worker Lambda → SES

Route 53 (DNS)

- **What:** AWS's managed DNS. It translates `events.college.edu` into addresses.
- **Why here:** an **ALIAS** record can point the domain apex at CloudFront (a normal CNAME can't be used at the apex). It's free to query for AWS targets and has a 100% availability SLA.
- **Alternative:** the college's existing DNS provider with a CNAME. That's fine too; Route 53 is optional in my Terraform.

CloudFront (CDN) and ACM

- **What:** a content delivery network with hundreds of edge locations worldwide, many in India. It caches content close to users and terminates HTTPS. ACM provides free, auto-renewing TLS certificates. For CloudFront, the certificate must be in **us-east-1**.
- **Why:**
  - It absorbs the page-load spike at the edge.
  - It gives one domain for site and API, so no CORS is needed.
  - It is the attachment point for WAF.
- **Design detail:** two *behaviours*. `/*` → S3 with caching on. `/api/*` → API Gateway with **caching off**, because seat counts must be live.
- **Alternative:** serve the site directly from S3 static website hosting. That's HTTP only, has no WAF and requires a *public* bucket, so I rejected it.

S3 (object storage)

- **What:** stores files (objects) in buckets. Designed for 99.999999999% durability.
- **Security:**
  - The bucket is **private**, with Block Public Access on.
  - **Origin Access Control (OAC)** lets only CloudFront read it, and the bucket policy checks the *specific* distribution ARN.

    - Versioning is on, so a bad upload can be rolled back.
- **S3 vs EBS vs EFS:**
    - **S3** is object storage accessed over HTTP.
    - **EBS** is a block disk attached to one EC2 instance.
    - **EFS** is a shared network file system for many instances.
    - Static website files belong in S3.

WAF (Web Application Firewall)

- **What:** inspects HTTP requests and blocks bad ones before they reach the app.
- **Rules:**
    - AWS managed **Common Rule Set** (OWASP-style: SQLi, XSS)
    - **Known Bad Inputs** rule set
    - a **rate-based rule** on `/api/*`
- **Reasoning detail (a great point to mention):** the per-IP limit is set *high* (2,000 per 5 min). A whole campus Wi-Fi often shares **one public IP through NAT**, so a strict per-IP limit would block every student at once during the rush. API Gateway's global throttle is the real safety valve.
- **Cost:** about $5/month per web ACL + $1 per rule. It's the only meaningful fixed cost, so it's optional in Terraform.

API Gateway (HTTP API)

- **What:** a managed front door for APIs. It handles routing, throttling, authorisation and access logs.
- **Why HTTP API rather than REST API:** cheaper (~70% less) and lower latency, and it has the JWT authoriser I need. A REST API adds usage plans, API keys, request validation and caching, none of which I need.
- **Throttling:** 200 req/s steady and 400 burst. Above that it returns `429` *without invoking Lambda*, protecting the backend and the bill.
- **Auth:** the `POST /api/events` route has a **JWT authoriser** that checks the Cognito token's signature, expiry, issuer and audience. Invalid tokens get `401` before Lambda runs.
- **Alternative:** Application Load Balancer → Lambda. An ALB is billed per hour (~$16+/month even when idle). It makes sense when you already have an ALB for containers.

Lambda (compute)

- **What:** runs code in response to events, billed per request and per millisecond. You don't manage servers.
- **Scaling:** each concurrent request gets its own execution environment. Functions scale by up to **1,000 concurrent executions every 10 seconds**, up to the account's regional concurrency quota (default **1,000**; new accounts can be lower, so check before an event).
- **Capacity maths (Little's law):** concurrency = arrival rate × duration. 100 req/s × 0.1 s = **10** concurrent executions. That's plenty of headroom.
- **Cold start:** the first request in a new environment loads the runtime and code: ~0.5–1.5 s for Python in a container. Warm environments are reused, which is why boto3 clients are created *outside* the handler (`@lru_cache` in `main.py`). Fixes if needed:
    - **provisioned concurrency**, scheduled only for the launch window

    - smaller images
- **Container image vs zip:** the image (up to 10 GB) gives one artefact used locally, in CI and in Lambda. A zip package is limited to 250 MB unzipped but cold-starts slightly faster.
- **Versions and alias:** each release is published as an immutable *version*. API Gateway always calls the alias `live`. Release = move alias to N; rollback = move alias back to N-1 in seconds.
- **Lambda vs EC2:**
    - **EC2** = you rent a VM: full control, always-on cost, you patch the OS and handle scaling.
    - **Lambda** = no servers, per-request cost, automatic scaling, but max 15 minutes per invocation and no persistent local state.
    - **ECS Fargate** = containers without managing VMs; a good middle ground for long-running services.

DynamoDB (database)

- **What:** fully managed NoSQL key-value/document database. Single-digit-millisecond latency, data replicated across 3 AZs.
- **Data model:**
    - `events` (PK `event_id`)
    - `registrations` (PK `event_id`, SK `email`). The composite key itself enforces "one registration per email per event".
- **On-demand mode:** pay per request and scale automatically. It handles up to **double the previous peak** instantly, and "warm throughput" can pre-warm a table before a known launch. **Provisioned** mode is cheaper for steady, predictable traffic, which this isn't.
- **The overbooking problem and solution (expect questions here):**
    - Naive approach: read count (2 left) → check → write. Two concurrent requests both read "1 left" and both write, so you've overbooked. This is a **race condition**.
    - Solution: `TransactWriteItems` with two conditional operations:
        1. `UpdateItem` on the event: `SET registered_count = registered_count + 1`, **condition** `registered_count < capacity`
        2. `PutItem` registration: **condition** `attribute_not_exists(email)`
    - It's **atomic** (all or nothing) and **conditional**, evaluated by DynamoDB itself, so no client-side race is possible.
    - Cost: transactions consume 2× capacity units. Worth it for correctness.
    - On failure, `CancellationReasons` tells us *which* condition failed, so we know whether it's "duplicate" or "full".
- **Consistency:** reads are *eventually consistent* by default (cheaper; may be ~1 s stale). Strongly consistent reads are available. The *write conditions* are always evaluated on the latest data, which is what guarantees correctness.
- **Backups:** point-in-time recovery (restore to any second in the last 35 days) and deletion protection.
- **DynamoDB vs RDS:** see §8.

SQS → worker Lambda → SES (asynchronous email)

- **SQS** is a fully managed message queue. It **decouples** producer and consumer, **buffers** bursts, and retries.

- **Why async:** sending email is slow and can fail. The student gets "Confirmed" in milliseconds; the email follows seconds later.
- **Visibility timeout** (180 s): while one worker processes a message, it's hidden from others. Set to ≥ 6× the function timeout.
- **Dead-letter queue (DLQ):** after 5 failed receives, the message is parked for investigation. An alarm fires on any DLQ message.
- **Partial batch failure:** the worker returns only the failed message IDs, so SQS retries just those. Without it, one bad message would make all 10 in the batch retry, sending duplicate emails.
- **At-least-once delivery:** standard SQS may deliver a message twice, so consumers should be **idempotent**. A rare duplicate email is acceptable here. FIFO queues give exactly-once *processing* with lower throughput.
- **SES** sends the email. **SES starts in a sandbox**: verified recipients only, ~200 emails/day. You must request production access before a real event. (Classic "emails don't arrive" gotcha.)
- **SNS vs SQS:** SNS is **pub/sub push** (one message fans out to many subscribers, nothing stored). SQS is a **pull queue** (one consumer processes each message; it's stored until handled). I use SNS for alarm notifications and SQS for work.

Cognito (identity)

- User pool for **organisers only**: admin-invite, 12-character passwords, optional MFA. It issues JWTs (ID/access tokens) valid for 1 hour.
- Students don't need accounts, to reduce friction. Validation is by input rules plus the uniqueness of email per event.
- **Defence in depth:** even after API Gateway validates the token, the app checks for the `organisers` group, so a misconfigured route can't silently become public.

IAM (permissions)

- **Users** (people), **roles** (assumed by services or CI, giving *temporary* credentials) and **policies** (JSON allow/deny on actions and resources).
- **Least privilege:** each function has its own role with only the exact actions on the exact ARNs. If the API code were compromised, it still couldn't delete tables or read other resources.
- **No long-lived keys:** Lambda uses its execution role, and GitHub Actions uses OIDC federation.

CloudWatch, X-Ray, SNS (observability)

See §4 *Monitoring* and §7 *Troubleshooting*.

Availability: what "highly available" means here

- **Region:** a geographic area (e.g. Mumbai), containing multiple **Availability Zones**. Each AZ is one or more data centres with independent power and networking, a few km apart.
- **Edge locations:** CDN points of presence; there are many more of them than regions.
- Lambda, DynamoDB, SQS, S3, API Gateway and Cognito are **regional, multi-AZ services**: AWS runs them across AZs, so an AZ failure is invisible to us.
- I did **not** go multi-region: it would double cost and complexity, and need a multi-region data strategy (DynamoDB Global Tables). For a college event site, a rare region outage is an accepted risk.

- **RTO** (recovery time objective) is hours: redeploy with Terraform elsewhere. **RPO** (data-loss window) is seconds to minutes via PITR or backups.

Why not a VPC?

Expect this question.

> "None of my components need one. Lambda talks to DynamoDB and SQS over AWS's authenticated, TLS-encrypted APIs using IAM. Putting Lambda in a VPC would only be needed for private resources like RDS or ElastiCache. It would then also need **NAT gateways** (~$32/month each, plus data charges) or **VPC endpoints** to reach AWS services. If I added RDS, I'd put it in **private subnets** across two AZs, with security groups allowing only the Lambda's security group on port 5432, and use VPC gateway endpoints for DynamoDB and S3."

Know the VPC vocabulary anyway:

- **VPC:** a private network (CIDR, e.g. 10.0.0.0/16).
- **Subnets:** each lives in one AZ. *Public* subnets have a route to an **Internet Gateway**. *Private* subnets reach out through a **NAT gateway**.
- **Route tables:** decide where each subnet's traffic goes.
- **Security groups:** *stateful*, instance-level, allow rules only.
- **Network ACLs:** *stateless*, subnet-level, allow and deny rules.

---

4. Task 3 deep dive: be ready to explain each line

Git and GitHub

- **commit** (snapshot with message) · **branch** (movable pointer) · **merge** (combine histories) · **rebase** (replay commits on a new base for a linear history; never rebase shared branches) · **pull request** (review + CI gate) · **tag** (named release)
- **Branching strategy:** *GitHub Flow*. Short-lived feature branches go to `main`, and `main` is always deployable. *Git Flow* (develop/release branches) is heavier and suited to versioned products. *Trunk-based development* uses very small, frequent merges, often with feature flags.
- **Merge conflict:** two branches changed the same lines. Git asks a human to choose, then you re-run tests.
- **Branch protection:** require PR review and passing checks, and disallow force-push to `main`.

Build and test

- **Test pyramid:** many fast unit tests, fewer integration tests, very few end-to-end tests.
  - Mine: 17 unit tests (seconds) → container smoke test → full-stack rush test.
- **moto** fakes AWS APIs in memory: fast, free and safe.
  - Limitation: it doesn't perfectly mimic AWS. That's why the rush test also runs against **DynamoDB Local**, and why a staging environment is the next step.
- **Coverage gate 80%:** coverage measures which lines ran, *not* whether behaviour is correct. It's a floor, not a goal.

- **Shift-left security:** `ruff`'s bandit rules on code and Trivy on the image catch issues before release instead of after.

CI/CD in my pipeline

- Jobs: `test` → `container` and `terraform` (run in parallel) → `deploy`.
- **CI vs Continuous Delivery vs Continuous Deployment:**
  - **CI:** integrate and test every change automatically.
  - **Delivery:** every green build *can* be released, with a human approval step.
  - **Deployment:** every green build *is* released automatically.
  - Mine is **Continuous Delivery**.
- **Deployment strategies:**
  - **All-at-once:** simple, but risky.
  - **Rolling:** replace instances gradually.
  - **Blue/green:** two full environments and an instant switch. **My alias switch is a blue/green-style cutover**: version N-1 is blue, N is green.
  - **Canary:** send e.g. 10% of traffic to the new version first. Lambda aliases support *weighted routing*, and AWS CodeDeploy can automate canary with alarm-based rollback. That's my next improvement.
  - **Feature flags:** deploy code switched off, then enable it without redeploying.
- **OIDC** (be able to explain step by step):
  1. The job requests a signed token from GitHub's OIDC provider. Its claims include repo, branch and environment.
  2. AWS STS `AssumeRoleWithWebIdentity` verifies the signature and checks the role's trust policy conditions (`aud = sts.amazonaws.com`, `sub = repo:<owner>/<repo>:environment:production`).
  3. STS returns credentials that expire in about an hour.

  Nothing long-lived exists to leak.
- **Secrets in CI:** use GitHub *secrets* for real secrets (masked in logs). I use only *variables*, because role ARNs and bucket names aren't secret.
- **Pipeline speed:** pip cache, Docker layer caching, parallel jobs, and cancelling superseded runs.
- **DORA metrics** (how teams measure delivery performance):
  - deployment frequency
  - lead time for changes
  - change failure rate
  - time to restore service

Docker

- **Image:** read-only template made of layers. **Container:** running instance of an image. **Dockerfile:** recipe. **Registry:** storage for images (ECR, Docker Hub).
- **Layers and cache:** each instruction is a layer. Put rarely-changing steps (installing dependencies) before frequently-changing ones (copying code).
- **Multi-stage builds:** separate build and runtime stages, or (in my case) targets for different environments, without duplicating the Dockerfile.
- **Container vs VM:** containers share the host OS kernel (lightweight, start in seconds). VMs each run a full guest OS on a hypervisor (stronger isolation, heavier).
- **.dockerignore:** keeps tests, caches and `.env` files out of the image.
- **Tagging:** use the **immutable Git SHA**, not `latest`. `latest` doesn't tell you what's running and can change under you.
- **Architecture gotcha:** Apple-silicon Macs build `arm64` images by default, while my Lambda is `x86_64`. Pushing a Mac-built image fails with `exec format error`, so use `docker build --platform linux/amd64`. (Or switch Lambda to arm64/Graviton, which is ~20% cheaper.)
- **docker compose:** defines multi-container apps: services, networks, healthchecks and `depends_on` conditions.

Terraform (IaC)

- **Declarative:** you describe the desired end state, and Terraform works out the steps. **Imperative** scripts list the steps themselves.
- **Commands:** `init` · `fmt` · `validate` · `plan` · `apply` · `destroy` · `import` (bring an existing resource under management) · `state list`
- **State:** the mapping between code and real resource IDs.
  - **Remote state** in S3 with **locking**: one source of truth, no concurrent applies.
  - Never edit state by hand, and never commit it (it may contain secrets).
- **Drift:** reality differs from the code (someone clicked in the console). `plan` shows it, and you either `apply` to revert it or update the code.
- **Providers** (AWS plugin, version pinned with `~> 6.0` and a lock file) · **variables** and **outputs** · **data sources** (look up existing things, e.g. managed CloudFront policies) · **modules** (reusable packages of resources) · `count` / `for_each` (optional or repeated resources: my custom domain uses `count`, alarms use `for_each`).
- **Lifecycle `ignore_changes`:** used deliberately so Terraform doesn't fight the pipeline over the image/version (explained in Task 3 §5).
- **Bootstrap ordering:** Lambda requires an existing image, but ECR is created by the same code. Solved with `-target` first. Know this story; it shows real-world experience.
- **Alternatives: CloudFormation** (AWS-native, no state file to manage, AWS-only) · **AWS CDK** (write infrastructure in Python/TS, compiles to CloudFormation) · **Pulumi** · **OpenTofu** (open-source fork of Terraform).

Monitoring and logs

- **Three pillars:** metrics (numbers over time), logs (events with detail), traces (one request's journey across services).

- **Structured logging:** JSON fields can be queried. Plain text needs fragile regexes.
- **Golden signals (Google SRE):** latency, traffic, errors, saturation. My alarms map to these:
  - p95 latency
  - request count
  - 5xx and Lambda errors
  - throttles and queue depth
- **Percentiles vs averages:** an average of 200 ms can hide 5% of users waiting 4 s. Alarm on p95/p99.
- **SLI / SLO / SLA:**
  - **SLI:** what you measure, e.g. % of registrations that succeed in under 1 s.
  - **SLO:** your target, e.g. 99.5% during registration windows.
  - **SLA:** a contractual promise with penalties.
- **Alert fatigue:** alarm only on user-visible symptoms. `treat_missing_data = notBreaching` stops "no traffic at 3 a.m." from paging anyone.
- **Liveness vs readiness:** `/health` says the process is up; `/health/ready` says it can do real work (reach the database).

---

5. Cloud fundamentals: rapid-fire Q&A

What is cloud computing? On-demand access to computing resources over the internet, paid for as you use them, without owning hardware. Key traits:

- self-service
- elasticity
- pay-as-you-go
- global reach
- managed services

IaaS vs PaaS vs SaaS vs FaaS?

Model	You manage	AWS example
IaaS	OS and up	EC2
PaaS	Just the app	Elastic Beanstalk, App Runner
SaaS	Just use it	Gmail
FaaS / serverless	Just functions	Lambda

Shared responsibility model?

- AWS secures the **cloud itself**: hardware, facilities, the hypervisor, managed-service internals.
- You secure **what you put in the cloud**: data, IAM, configuration, encryption choices, your code.
- The more managed the service, the more AWS takes on. With Lambda you don't patch the OS; with EC2 you do.

Vertical vs horizontal scaling?

- Vertical = a bigger machine. Simple, but has a ceiling and usually needs downtime.

- Horizontal = more machines behind a load balancer. Near-unlimited, but the app must be **stateless**.

Why does statelessness matter? If any server or Lambda can handle any request, you can add or remove them freely. Keep state in a database or cache, not in memory or on local disk.

High availability vs fault tolerance vs disaster recovery?

- **HA:** minimise downtime, e.g. multi-AZ with fast failover.
- **Fault tolerance:** keep working with zero interruption when a component fails.
- **DR:** recover from a large-scale disaster (region loss), measured by RTO and RPO.
- DR strategies, from cheapest to fastest: backup & restore → pilot light → warm standby → multi-site active/active.

What does a load balancer do? Distributes traffic across healthy targets, health-checks them, and removes failed ones. It can terminate TLS.

- **ALB** works at layer 7 (HTTP; path- and host-based routing).
- **NLB** works at layer 4 (TCP/UDP; extreme performance, static IPs).

What is a CDN? What gets cached? Edge servers that cache content near users. Cache static assets with long TTLs. Use versioned filenames or **invalidations** when content changes. Never cache personalised or live data, unless it's short "micro-caching" you have deliberately accepted.

Walk me through what happens when I type a URL.

1. DNS resolution (browser cache → OS → resolver → root → TLD → authoritative, e.g. Route 53) returns an IP address.
2. TCP connection, then TLS handshake (certificate verified against trusted certificate authorities).
3. HTTP request is sent, and the server or CDN responds.
4. The browser renders the page and fetches further assets.

DNS record types?

- **A:** name → IPv4 address. **AAAA:** name → IPv6 address.
- **CNAME:** name → another name (not allowed at the zone apex).
- **ALIAS** (Route 53): like a CNAME, but allowed at the apex and pointing at AWS resources.
- **MX:** mail. **TXT:** verification and SPF/DKIM. **NS:** delegation.
- **TTL:** how long resolvers cache the answer.

HTTP status codes you should know?

Code	Meaning
200	OK
201	Created
301 / 302	Redirect
304	Not modified
400	Bad request
401	Not authenticated

Code	Meaning
403	Authenticated but not allowed
404	Not found
409	Conflict (I use it for full or duplicate)
422	Validation error
429	Too many requests
500	Server bug
502	Bad gateway (upstream gave an invalid response)
503	Unavailable
504	Gateway timeout

SQL vs NoSQL?

- **SQL:** fixed schema, joins, transactions, flexible queries. Scales vertically, plus read replicas.
- **NoSQL** (e.g. DynamoDB): design the data around the access patterns, horizontal scale, predictable performance, limited ad-hoc queries.
- Choose by access patterns and consistency needs, not hype.

RDS Multi-AZ vs read replicas?

- **Multi-AZ:** a synchronous standby in another AZ, used for **availability** (automatic failover). It doesn't serve reads in the classic setup.
- **Read replicas:** asynchronous copies used to **scale reads**. They can be promoted for disaster recovery.

Encryption at rest vs in transit?

- **At rest:** data stored encrypted, with keys managed by **KMS**.
- **In transit:** TLS on the wire.
- My design uses both everywhere.

What is KMS? A managed service that creates and controls encryption keys. Services use **envelope encryption**: data keys encrypted by a KMS key. Access is controlled by IAM and key policies, and every use is audited in CloudTrail.

What is CloudTrail vs CloudWatch?

- **CloudTrail:** an audit log of *API calls*. Who did what, when, from where.
- **CloudWatch:** *operational* metrics, logs and alarms. How the system is behaving.

How do you control cloud costs?

- AWS Budgets with alerts, and cost allocation tags (I tag every resource via `default_tags`).
- Right-sizing, serverless or auto-scaling for spiky loads.
- Turn off idle resources, and set log retention limits.
- Watch hidden costs: NAT gateways, data transfer out, idle load balancers.

How do you secure an AWS account?

- MFA on root, then never use root day to day.
- Individual identities (IAM Identity Center / SSO) and least privilege.
- No access keys in code, and rotate any that exist.
- CloudTrail on, with budget alerts.
- Block public S3 access by default.
- GuardDuty and Security Hub for threat detection.

Why must API keys and passwords never be public? Bots scan GitHub for leaked AWS keys within minutes, and a leaked key can run up thousands of dollars in crypto-mining or expose data. Precautions:

- use roles or OIDC instead of keys
- keep secrets in Secrets Manager or Parameter Store
- `.gitignore` your `.env` files
- secret scanning (GitHub push protection)
- MFA, and immediate rotation if a key leaks

---

6. DevOps: rapid-fire Q&A

What is DevOps? A culture and set of practices that unite development and operations. Small frequent changes, automation (CI/CD, IaC), shared ownership of production, fast feedback from monitoring, and blameless learning from failure. It isn't a job title or a tool.

Why small, frequent deployments? Smaller changes are easier to review, test and roll back, and when something breaks the cause is obvious. Big-bang releases bundle many risks together.

How would you add a staging environment? Run the same Terraform with a different `environment` variable and its own state key (or a separate AWS account; separate accounts are best practice). The pipeline deploys to staging automatically, runs integration and load tests, then promotes **the same image** to production after approval. Never rebuild for production.

What's immutable infrastructure? Never patch a running server. Build a new image or version and replace the old one. My immutable ECR tags and Lambda versions are examples. It makes rollbacks reliable and environments identical.

Configuration vs code; where do secrets go?

- Configuration goes in environment variables (Twelve-Factor App). The same image runs everywhere, and only the environment differs.
- Secrets go in Secrets Manager or SSM Parameter Store, fetched at runtime with IAM, never baked into images or committed.

How do you roll back a database change? You often can't simply "roll back" data. Use **backward-compatible migrations** (expand/contract):

1. Add a new column.
2. Deploy code that writes both the old and new columns.

3. Backfill existing data.
4. Switch reads to the new column.
5. Remove the old column later.

Keep backups or PITR as a last resort.

What is GitOps? Git is the source of truth for desired state, and an automated agent continuously reconciles the live system to match it (e.g. Argo CD for Kubernetes). Every change goes through a PR.

Kubernetes in one breath (not required, but in case you're asked)? A container orchestrator. You declare desired state: Deployments of Pods, Services for networking, Ingress for HTTP routing. It schedules containers onto nodes, restarts failed ones, scales them and rolls out updates. On AWS it's EKS. Overkill for this project: serverless gives the same benefits with no cluster to run.

---

7. Troubleshooting (15% of the score)

A method to say out loud

> "First I'd **scope** it: who's affected, since when, and *what changed*. Then I'd **look at metrics** to see the shape of the problem, **logs** for the specific error, and **traces** for where the time goes. I'd form a **hypothesis**, test it in the least risky way, **mitigate first** (for example roll back), then fix the root cause, **verify** with the same metrics, and write a **blameless post-mortem** with a test or alarm so it can't happen silently again."

The key phrase is *"what changed?"*. Most incidents follow a deploy or a config change, and rollback is often the fastest mitigation.

Scenarios

1. Right after a deploy, students get HTTP 500 on registration.

- *Look:* CloudWatch 5xx and Lambda `Errors` alarms fire. In Logs Insights, filter `status >= 500` and read the exception.
- *Mitigate:* move the `live` alias back to the previous version (seconds).
- *Example root cause, which actually happened in my project:* `ValidationException: reserved keyword: capacity` in a DynamoDB expression. Fix with `ExpressionAttributeNames`, and add the test that would have caught it.

2. Students see "Too many requests" (429) during the rush.

- It comes from one of three places: **WAF** (rate rule), **API Gateway** (stage throttle) or **Lambda** (concurrency limit, showing as throttles).
- *Check:* WAF sampled requests, API Gateway `4xx` / `429` metrics, Lambda `Throttles` and `ConcurrentExecutions`.
- *Fix:*
  - Raise the stage throttle.
  - Request a Lambda concurrency quota increase *before* the event.

    - Make sure the WAF per-IP limit isn't hitting the campus NAT IP.
    - Frontend: retry with exponential backoff.

3. "Event full" is shown, but the organiser says seats remain.

- Check the item in DynamoDB: compare `registered_count` vs `capacity`. Then check whether the frontend is showing a cached response: is CloudFront caching `/api/*`? It must use `CachingDisabled`.
- Also check whether failed or abandoned registrations were counted. In my design they can't be: the transaction is all-or-nothing.

4. The website shows the old version after a deploy.

- CloudFront is still caching the old files. Check the `create-invalidation` step ran and completed. Long term, use content-hashed filenames (`app.3f9c.js`) so new files never collide with cached ones.
- Also check the browser cache (hard refresh) and that `s3 sync` actually uploaded the files.

5. Confirmation emails aren't arriving. Follow the pipeline: API logs ("Failed to enqueue"?) → SQS queue depth (stuck?) → worker logs (errors?) → **DLQ** (messages parked?) → SES.

- **Most likely cause: SES sandbox**, which only sends to verified addresses. Request production access.
- Also check: unverified sender identity, spam folder, and SPF/DKIM DNS records.
- After fixing, **redrive** DLQ messages back to the main queue.

6. Lambda logs show `AccessDeniedException ... not authorized to perform dynamodb:UpdateItem`.

- The IAM role lacks the action, or the policy points at the wrong table ARN.
- Read the exact action and resource in the error message, fix the Terraform policy (keep least privilege, no `*`), apply, and retest.
- CloudTrail shows the denied call too.

7. Requests time out (API Gateway 504 / Lambda `Task timed out`).

- Check the Lambda duration metric and X-Ray trace to find which call is slow.
- Usual suspects:
    - a downstream service is slow and the client has no timeout set
    - too little memory (CPU scales with memory)
    - cold starts
    - (if in a VPC) no NAT or VPC endpoint, so AWS API calls hang
- Remember API Gateway's integration timeout is ~30 s, so the Lambda timeout should be lower so it fails fast.

8. My image works locally but Lambda fails with `exec format error`. CPU architecture mismatch: arm64 (Apple silicon) vs x86_64 (Lambda config). Rebuild with `--platform linux/amd64`, or switch Lambda to `arm64`.

9. `terraform apply` fails: "Source image ... does not exist". Bootstrap ordering: Lambda needs an image that isn't in ECR yet. Apply the ECR repository with `-target`, push the bootstrap image, then run the

full apply.

10. `terraform apply` says the state is locked.

- Another apply is running (CI or a teammate). Wait.
- If a crashed run left the lock behind, confirm nobody is applying, then run `terraform force-unlock <id>`. Never delete state files by hand.

11. The browser console shows a CORS error.

- The page origin differs from the API origin and the API didn't send `Access-Control-Allow-Origin`.
- My design avoids this by serving both on one domain through CloudFront.
- If you call the API Gateway URL directly, configure CORS on the HTTP API with explicit origins, not `*` for credentialed calls.

12. CI fails with "coverage below 80%", or the Trivy scan fails.

- Coverage: add tests for the new code. Don't lower the gate.
- Trivy: read the CVE. If fixed upstream, rebuild with an updated base image or dependency. If not fixable and not exploitable in your context, document the exception and its expiry.
- Know that my image currently has **0 fixable CRITICAL and ~30 HIGH** OS-package findings. I'd add a scheduled weekly rebuild to pick up patched base images.

13. The site is slow only for the first request after a quiet period. Cold start. Measure `Init Duration` in the Lambda `REPORT` log lines. Mitigate by:

- keeping work outside the handler minimal
- trimming dependencies
- using provisioned concurrency during the event window

A post-mortem outline (blameless)

1. Summary and impact (who, how long).
2. Timeline.
3. Root cause: *why* it happened, and why it wasn't caught (the "5 whys").
4. What went well and what went badly.
5. Action items with owners, e.g. "add test", "add alarm", "update runbook".

---

8. Defending trade-offs (likely follow-up questions)

"Why not EC2?"

- EC2 means always-on cost, OS patching and slower scaling for a spike that lasts minutes.
- I'd choose EC2 or ECS for long-running processes, WebSockets, heavy CPU work over 15 minutes, or when I need full OS control.

"Why not RDS / PostgreSQL?"

- My access patterns are key lookups, and correctness comes from a conditional transaction.

- DynamoDB scales on demand without connection limits. Lambda × many concurrent connections can exhaust RDS, which needs RDS Proxy.
- If the club needed rich reports, I'd use Aurora Serverless v2, or export DynamoDB to S3 and query it with Athena.

"What if 10× the students show up?" 1,000 req/s × 0.1 s ≈ 100 concurrent Lambdas, still under the default quota. DynamoDB on-demand and API Gateway handle it. I'd:

- raise API Gateway throttles
- confirm quotas
- run a load test in staging (e.g. k6 or Locust)
- possibly add 2–5 s micro-caching on `GET /api/events`

"What's the single biggest risk in your design?" Account-level quotas and SES sandbox limits are *configuration* risks, not architecture risks. That's why my pre-event checklist includes:

- check Lambda concurrency quota
- SES production access
- a load test
- alarm subscriptions confirmed

"Someone could call your API Gateway URL directly and skip WAF." Correct; it's a known gap. Fix:

- CloudFront adds a secret custom header (stored in Secrets Manager and rotated), and the API rejects requests without it.
- Or put WAF on the API too. (HTTP APIs can't attach WAF directly; a REST API or ALB can.)

"Isn't serverless vendor lock-in?" Partly.

- I reduced it: the app is plain FastAPI in a standard container, and only the thin Mangum adapter is Lambda-specific. The same image runs under uvicorn anywhere.
- The data layer is the more locked-in part.
- For a student club, the operational savings outweigh the lock-in.

"How would you make the email step bulletproof?" The **transactional outbox** pattern. Write a "pending email" item *in the same DynamoDB transaction* as the registration. Then **DynamoDB Streams** triggers the email worker. Now the email can't be lost between the database write and the queue.

"What would you do differently with more time?"

1. Staging environment and a load test in the pipeline.
2. Canary releases with automatic alarm-based rollback (CodeDeploy).
3. `terraform plan` on PRs, using a read-only role.
4. Outbox pattern.
5. Waitlist feature.
6. arm64/Graviton.
7. Delete student data automatically after the event (DynamoDB TTL), for privacy.

---

9. Numbers worth remembering

Fact	Value
Lambda max timeout / image size	15 min / 10 GB
Lambda scaling rate	+1,000 concurrent executions per 10 s, per function
Default regional Lambda concurrency quota	1,000 (can be lower for new accounts)
API Gateway integration timeout	~30 s
DynamoDB on-demand	instantly absorbs up to 2× previous peak
DynamoDB transaction	up to 100 items; 2× capacity cost
DynamoDB PITR window	35 days
SQS visibility timeout guidance for Lambda	≥ 6× function timeout
S3 durability	11 nines (99.999999999%)
SES sandbox	verified recipients only, ~200 emails / 24 h
NAT gateway	~US$32/month + data, the classic hidden cost
WAF	~US$5/ACL + US$1/rule per month + per-request fee

10. Repo tour (if they ask "show me")

1. `backend/app/repository.py`: the `register()` transaction and how a failure is mapped to "full" vs "duplicate".
2. `backend/tests/test_api.py`: `test_full_event_rejects_and_never_overbooks`.
3. `scripts/concurrency_test.py`: run it live (`docker compose up`, then the script).
4. `backend/Dockerfile`: base image, layer caching, the `local` vs `lambda` targets.
5. `.github/workflows/ci-cd.yml`: jobs, OIDC, publish → smoke test → alias switch → rollback.
6. `infra/api.tf`: IAM least privilege, alias, throttling, JWT route; `infra/github_oidc.tf`: the trust conditions.
7. `infra/monitoring.tf`: alarms, the log metric filter, the dashboard.

Sources

- AWS Well-Architected Framework (six pillars) — https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html
- Overview of AWS (regions, AZs, services) — https://docs.aws.amazon.com/whitepapers/latest/aws-overview/introduction.html
- Lambda scaling and concurrency — https://docs.aws.amazon.com/lambda/latest/dg/scaling-behavior.html
- Lambda container images — https://docs.aws.amazon.com/lambda/latest/dg/images-create.html
- Lambda aliases — https://docs.aws.amazon.com/lambda/latest/dg/configuration-aliases.html
- SQS error handling for Lambda — https://docs.aws.amazon.com/lambda/latest/dg/services-sqs-errorhandling.html

- DynamoDB transactions — https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/transaction-apis.html
- DynamoDB reserved words — https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/ReservedWords.html
- DynamoDB on-demand — https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/on-demand-capacity-mode.html
- CloudFront + S3 Origin Access Control — https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-restricting-access-to-s3.html
- API Gateway HTTP API JWT authorizers — https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-jwt-authorizer.html
- WAF rate-based rules — https://docs.aws.amazon.com/waf/latest/developerguide/waf-rule-statement-type-rate-based.html
- SES production access — https://docs.aws.amazon.com/ses/latest/dg/request-production-access.html
- Shared responsibility model — https://aws.amazon.com/compliance/shared-responsibility-model/
- GitHub Actions OIDC with AWS — https://docs.github.com/en/actions/security-for-github-actions/security-hardening-your-deployments/configuring-openid-connect-in-amazon-web-services
- Terraform S3 backend — https://developer.hashicorp.com/terraform/language/backend/s3
- Docker multi-stage builds — https://docs.docker.com/build/building/multi-stage/
- Google SRE book: Monitoring Distributed Systems — https://sre.google/sre-book/monitoring-distributed-systems/
- DORA four key metrics — https://dora.dev/guides/dora-metrics-four-keys/